

## OS Mod 5

---

### File System Structure

Logical Structure: The OS organizes files logically to represent a hierarchical directory structure. This structure is often a tree, where directories act as nodes and files as leaf nodes. Common structures include:

- Single-Level Directory: All files are stored in one directory (limited and less common).
- Two-Level Directory: A separate directory for each user, offering basic user isolation.
- Hierarchical (Tree-Structured) Directory: Nested directories allow subdirectories within directories, widely used in modern OSs.
- Acyclic Graph Directory: Allows shared files and directories through links.
- General Graph Directory: Supports a more complex, networked approach (e.g., symbolic and hard links).

Physical Structure: Data is organized in blocks, and the OS maps files to these blocks on physical storage (e.g., HDD, SSD). Data structure techniques used here include:

- Contiguous Allocation: Files occupy consecutive blocks (fast but suffers fragmentation).
- Linked Allocation: Blocks are linked in a list (no fragmentation but slower access).
- Indexed Allocation: Index blocks hold pointers to file blocks, allowing random access without fragmentation.
- Multi-Level Indexed Allocation: Utilizes multiple layers of indexing for large files (e.g., in UNIX file systems).

**Directory Management:** Directories are specialized files that organize other files and provide a namespace for file lookup. They maintain metadata about files, like:

- File names, types, and locations on storage.
- Attributes (size, owner, permissions, etc.).
- Links or pointers to access files.

**File Control Blocks (FCBs):** FCBs (also known as inodes in UNIX systems) store details about each file, including attributes, pointers to data blocks, and access control information. When a file is accessed, the OS loads the FCB into memory to quickly manage file operations like reading, writing, and metadata retrieval.

### Access Methods

The file system provides various access methods, depending on how data is structured:

- Sequential Access: Data is read or written in a linear sequence, common in text and binary files.
- Direct (Random) Access: Allows accessing data at any point within the file, ideal for databases and applications that require quick lookups.

- **Indexed Access:** Combines direct access with index structures, typically used in large or structured data files.

**Space Management:** To optimize storage usage, the OS implements various techniques:

- **Free Space Management:** Uses bitmaps, linked lists, or free-space tables to track unused blocks.
- **Defragmentation:** Rearranges file blocks to enhance performance in contiguous or linked allocation systems.

**File System Interface and APIs:** The OS provides a file system interface that includes a set of system calls for file operations:

- **File Creation, Opening, Closing, Reading, Writing, and Deleting:** These are essential for interacting with files.
- **File Attributes and Permissions Management:** Controls access using permission bits or ACLs (Access Control Lists).
- **Path and File Navigation:** Commands like `cd`, `ls`, `pwd` (Linux), or `dir` (Windows) help users navigate the file system.

**Security and Permissions:** The OS uses security models (like DAC, MAC) and user-based permissions to control file access. Files are generally assigned a set of permissions based on owner, group, and others, with attributes like read, write, and execute access rights.

**File System Mounting:** Mounting links a file system to the directory structure, making it accessible under a directory (called a mount point). This process allows multiple file systems to be used concurrently.

**Secondary Storage Management:** Secondary storage (such as hard disks, SSDs, or external storage) is used for long-term data retention, unlike primary storage (RAM) that is volatile. Managing secondary storage effectively is essential for data retrieval speed, efficient use of space, and system performance.

**Storage Allocation Methods**

- **Contiguous Allocation:** Stores a file in consecutive blocks on the disk. It's fast for sequential access but can lead to fragmentation and is hard to expand.
- **Linked Allocation:** Uses a linked list of disk blocks, with each block containing a pointer to the next. This method prevents fragmentation but can be slower for random access.
- **Indexed Allocation:** Uses an index block with pointers to the file's blocks. This allows fast random access, as the OS can directly access any block.
- **Multi-Level Indexed Allocation (e.g., UNIX inodes):** Uses a hierarchy of index blocks for larger files, combining direct, indirect, and double-indirect indexing for scalability.

## Free Space Management

- Bitmaps or Bit Vectors: Uses bits to represent each block's status (0 for free, 1 for occupied). It's compact and fast for free space tracking.
- Linked List of Free Blocks: Links all free blocks, reducing memory requirements but making random access to free blocks slower.
- Grouping and Counting: Groups or counts free blocks to make tracking faster, particularly for large storage devices.

## Windows File Systems

### FAT (File Allocation Table)

Design: FAT is one of the simplest file systems, initially developed for floppy disks and early PCs. It organizes files with a File Allocation Table, which maps files to physical disk blocks.

#### Advantages:

- Simplicity makes it easy to implement and fast on older hardware.
- Universally supported, compatible with almost all OSs.

#### Disadvantages:

- Poor scalability; file and partition sizes are very limited.
- No security features like file permissions, making it less secure.
- No journaling, so it's vulnerable to data corruption if interrupted.

Use Case: Best suited for small, portable storage devices like USB drives or SD cards where compatibility is crucial.

### FAT32 (File Allocation Table 32)

Design: FAT32 expands on FAT by using 32 bits for the File Allocation Table entries, increasing partition and file size limits.

#### Advantages:

- Retains wide compatibility across devices, including Windows, Linux, and macOS.
- More efficient for small files than FAT.

#### Disadvantages:

- File size limit of 4 GB and partition limit of 2 TB.
- No journaling or advanced features like compression or encryption.

Use Case: Commonly used on flash drives and SD cards where cross-compatibility and ease of use are prioritized over advanced features.

### NTFS (New Technology File System)

Design: NTFS was designed for enterprise environments, with a focus on security, reliability, and support for large volumes.

Key Features:

- Journaling: Keeps track of changes, allowing quick recovery after crashes.
- Access Control Lists (ACLs): Allows fine-grained control over file permissions.
- Encryption and Compression: Supports EFS (Encrypting File System) and file compression.
- Sparse Files: Efficiently manages large files with empty data sections.
- Volume Shadow Copy: Allows creating point-in-time snapshots for backups.

Advantages:

- Highly reliable, especially in enterprise settings.
- Handles large files and volumes efficiently.

Disadvantages:

- Limited compatibility outside Windows.
- Overhead for small files compared to FAT-based systems.

Use Case: Preferred file system for Windows installations, particularly in enterprise or high-security environments where reliability and permissions management are crucial.

### **ReFS (Resilient File System)**

Design: Built to address the limitations of NTFS in terms of scalability and resilience, ReFS was designed for enterprise applications, data centers, and virtual machines.

Key Features:

- Data Integrity: Uses checksums for metadata and optional user data to detect and repair corruption.
- Automatic Error Correction: Real-time detection and recovery from data corruption.
- Scalability: Supports extremely large volumes and files.
- Storage Spaces Integration: Works well with Storage Spaces for pooling and mirroring.

Advantages:

- Provides fault tolerance and self-repair features, ideal for virtualized environments.
- Superior scalability compared to NTFS, especially for large data volumes.

Disadvantages:

- Limited compatibility; cannot be used as a bootable file system.
- Fewer features than NTFS (e.g., lacks compression, file-level encryption).

Use Case: Primarily used in server environments, large-scale storage, and situations where data resiliency is prioritized over backward compatibility.

## **Linux File Systems**

### ext2 (Second Extended File System)

Design: The first Linux file system to support large files and partitions, ext2 was known for simplicity and reliability.

Key Features:

- Simple structure with minimal overhead.
- No journaling, making it fast for write-heavy applications.

Advantages:

- Low overhead for small devices and boot partitions.
- Ideal for removable storage devices due to its efficiency and simplicity.

Disadvantages:

- Lack of journaling makes it prone to corruption if power is lost during write operations.

Use Case: Suitable for bootable USB drives and low-write environments; some embedded systems still use ext2 for simplicity.

### ext3 (Third Extended File System)

Design: Ext3 builds on ext2 by adding journaling, which provides resilience against corruption.

Key Features:

- Journaling: Writes changes to a log, which speeds up recovery after a crash.
- Three Journaling Modes: Ordered, writeback, and journal, allowing the user to prioritize data integrity or performance.
- Backward Compatibility: ext3 file systems can be mounted as ext2.

Advantages:

- Much faster recovery than ext2, making it suitable for critical applications.
- Minimal fragmentation compared to FAT-based file systems.

Disadvantages:

- Slower than ext4 and lacks advanced features.

Use Case: Widely used in Linux distributions due to stability and reliability; still seen in some legacy systems.

### ext4 (Fourth Extended File System)

Design: Ext4 is a high-performance, scalable system built on ext3's foundation with new features for speed and capacity.

Key Features:

- Extents: Efficient storage of large files by grouping contiguous blocks.
- Delayed Allocation: Helps prevent fragmentation by delaying allocation until data is actually written.
- Larger Volumes and Files: Can handle volumes up to 1 EB and files up to 16 TB.

Advantages:

- Offers better performance, especially for large volumes.
- Reduced fragmentation makes it ideal for general-purpose use.

Disadvantages:

- Not as fast as XFS for certain workloads, like high I/O.

Use Case: The default in many Linux distributions, ideal for a range of applications from general use to servers.

### ReiserFS

Design: Known for its efficiency with small files, ReiserFS employs B+ tree indexing for fast data access.

Key Features:

- Tail Packing: Reduces wasted space by packing small files or end-of-file data into block tails.
- Efficient Metadata Management: good for handling directories with 1000s of files.

Advantages:

- Optimized for workloads with numerous small files.
- Fast, especially for operations on directories with large numbers of files.

Disadvantages:

- Limited support and development since it is no longer actively maintained.

Use Case: Used in older Linux systems and specific applications needing small-file efficiency.

### XFS (Extended File System)

Design: Developed by SGI, XFS is optimized for high-performance computing with a focus on high throughput and scalability.

Key Features:

- B+ Tree Indexing: Speeds up access for large directories and files.
- Efficient Allocation: Uses delayed allocation to minimize fragmentation.
- Scalability: Supports large files and volumes, optimized for systems with large I/O requirements.

Advantages:

- Excellent performance for high-bandwidth applications.
- Handles large files and concurrent writes very efficiently.

Disadvantages:

- May be slower for workloads with many small files.
- Recovery can be slower than ext4 due to its journaling model.

Use Case: Commonly used in servers and high-performance computing environments, especially where large files and high I/O are typical.

## JFS (Journaled File System)

Design: IBM's JFS was created to minimize CPU usage and support large file systems.

Key Features:

- Efficient Journaling: Reduces the load on CPU by logging only metadata changes.
- Scalability: Suitable for handling large files and partitions.

Advantages:

- Low CPU usage makes it ideal for systems with limited resources.
- Quick recovery due to its efficient journaling mechanism.

Disadvantages:

- Fewer features than ext4 or XFS.
- Less commonly used and supported in Linux.

Use Case: Useful in environments where CPU efficiency is prioritized, though it has largely fallen out of favor in recent years.